Concrete Crack Detection Using Transfer Learning

Erwin S. De Jesus II
Artificial Intelligence and Machine Learning Engineer

I. Introduction

Concrete is one of the most widely used construction materials, and its structural integrity depends on early detection of cracks. Manual inspection is labour-intensive and subjective. This project develops a deep learning model to automatically classify images of concrete surfaces as cracked or non-cracked using transfer learning with the VGG16 architecture. The model is trained on the publicly available Concrete Crack Images for Classification dataset from Kaggle.

II. Dataset

The dataset contains 40,000 images evenly split into two classes:

Positive (with crack)	20,000 images
Negative (no crack)	20,000 images

Table 2.1 Content of the dataset

2.1 Data Splitting

To follow the same evaluation strategy used in a classification project, the dataset was split into three subsets using train_test_split from scikit-learn:

- Training (70%): 14,000 no_crack + 14,000 crack= 28,000 images
- Validation (15%): 3,000 no_crack + 3,000 crack = 6,000 images
- Test (15%): 3,000 no_crack + 3,000 crack = 6,000 images

Splitting the 40,000 images took 1 minute 49 seconds and created a new directory structure compatible with Keras ImageDataGenerator.

III. Methodology

3.1 Data Preprocessing

Images were resized to 224×224 pixels (the input size expected by VGG16) and pixel values were normalised to [0,1] by rescaling (1./255). Data generators were created for each split:

- Training generator: shuffle=True to randomise batches.
- Validation generator: shuffle=False for consistent evaluation during training.
- Test generator: initially shuffle=False, later changed to True for visualisation of mixed batches.

3.2 Transfer Learning with VGG16

The VGG16 model pre-trained on ImageNet was used as the feature extractor. The top classification layer was removed, and the convolutional base was frozen to retain the learned features.

3.3 Custom Classifier Head

On top of the frozen base, the following layers were added:

- Flatten()
- Dense(512, activation='relu')
- Dropout(0.3)
- Dense(512, activation='relu')
- Dropout(0.3)
- Dense(1, activation='sigmoid') for binary classification.

3.4 Model Compilation

- Optimizer: Adam with learning rate = 0.0001
- Loss function: binary_crossentropy
- Metric: accuracy

IV. Training

The model was trained for 5 epochs with a batch size of 32. Training was performed on the full 28,000-image training set, with validation on 6,000 images. The total training time was 237 minutes (approximately 4 hours).

Training Progress:

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.9885	0.0348	0.9955	0.0139
2	0.9950	0.0166	0.9958	0.0131
3	0.9962	0.0125	0.9972	0.0085
4	0.9974	0.0089	0.9972	0.0080
5	0.9971	0.0076	0.9973	0.0078

Table 4.1 Training Progress during model fitting

Both training and validation loss decreased rapidly and linearly over the epochs, indicating effective learning. Validation accuracy reached 99.73% by epoch 5, with negligible gap between training and validation which is a sign of excellent generalization and no overfitting.

V. Results

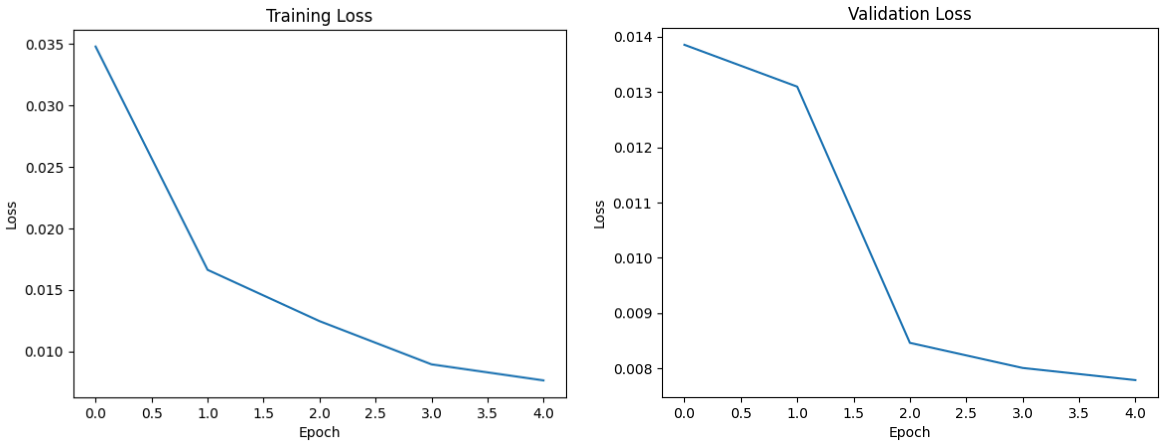

Figure 5.1 Loss Curves for Training and Validation

Figure 5.1 shows the plot of the loss during training and validation. This remarks that the loss gradually decreased linearly along with the number of epochs, indicating effective learning of the model during training.

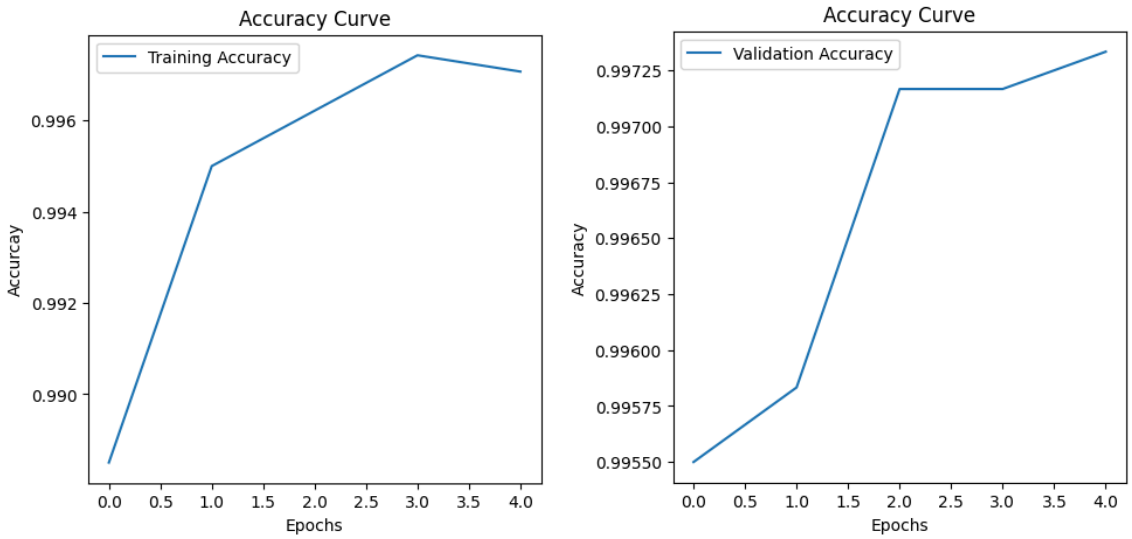

Figure 5.2 Accuracy Curves for Training and Validation

Figure 5.2 shows the plot of the accuracy during training and validation. This remarks that the accuracy gradually increased linearly along with the number of epochs, with validation accuracy reaching 99.73% exhibiting negligible gap between training and validation. This shows excellent generalization and no signs of overfitting.

5.2 Test Set Evaluation

After training, the model was evaluated on the held-out test set (6,000 images):

- **Test Loss: 0.0072**
- **Test Accuracy: 99.65%**

These metrics confirm that the model performs exceptionally well on unseen data, misclassifying only about 21 out of 6,000 images.

5.3 Visualizing Predictions

Initial visualisation using the test generator with shuffle=False showed only crack images, because the generator returns images in alphabetical folder order. After setting shuffle=True, mixed batches were obtained, and the model correctly classified both cracked and non-cracked samples.

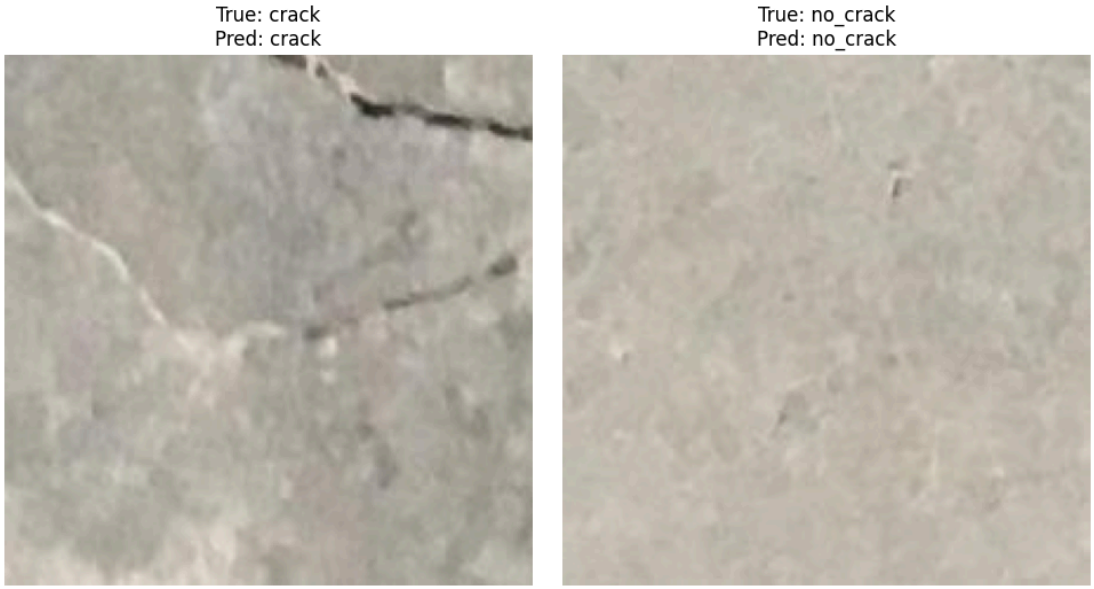

Figure 5.3 Sample Predictions using the Test dataset

Sample predictions show that the model correctly classifies many images, though misclassifications occur, especially on ambiguous damage patterns.

5.4 Testing on External Images

The model was also tested on completely new images (not part of the dataset):

- A crack image was correctly classified with a confidence of 0 (interpreted as class crack after adjusting the class mapping).
- A non-crack image was correctly classified with a confidence of 1.

The near-certain probabilities indicate that the model is highly confident in its predictions, which is consistent with the high test accuracy.

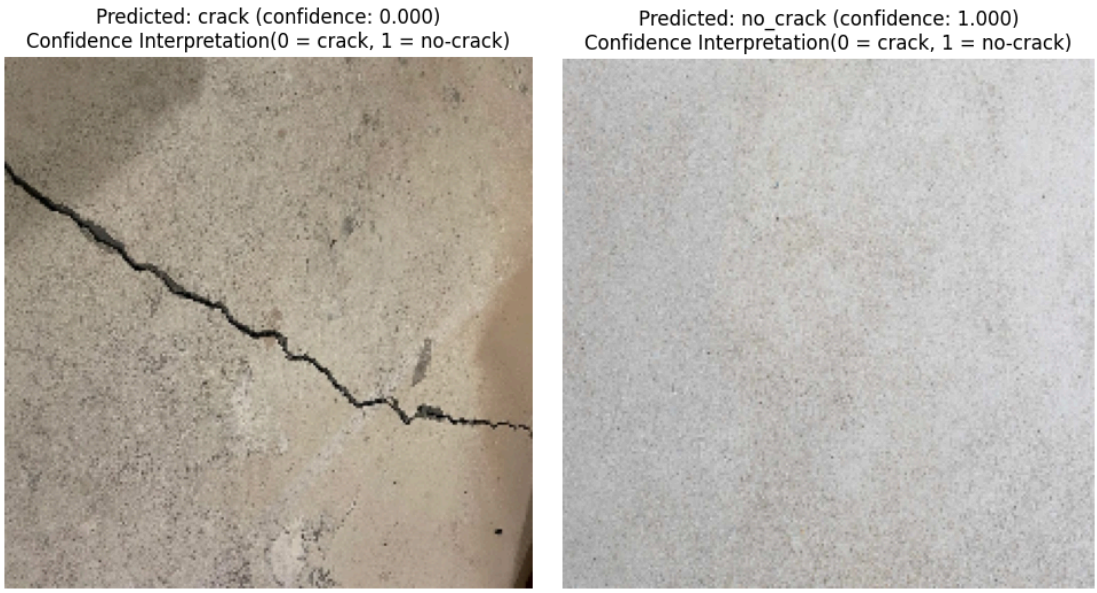

Figure 5.4 Predictions Using External Images (not part of the dataset)

VI. Discussion

Transfer learning effectiveness: Even with a relatively small custom head, the pre-trained VGG16 features enabled the model to achieve near-perfect accuracy with only 5 epochs.

Training time: The 4-hour training time reflects the large dataset (28,000 images × 5 epochs × 3 seconds per step). This is acceptable for a one-time training and can be reduced with hardware acceleration (GPU) or optimised data pipelines.

Class mapping clarification: The confidence values of 0 and 1 on external images highlighted the importance of correctly interpreting the model's output. After verifying the generator's class_indices, the mapping was confirmed as 0 → crack, 1 → no_crack, which matched the observed predictions.

VII. Conclusion

This project successfully developed a highly accurate concrete crack classifier using transfer learning with VGG16. The model achieved **99.65% test accuracy** and demonstrates excellent generalisation on new images. The results show that deep learning, even with a relatively simple transfer learning approach, can reliably automate the detection of structural cracks, offering a valuable tool for infrastructure maintenance.

VIII. Recommendations

Future improvements could include:

- Exploring integrated stacking ensembles to predict what type of crack (from ACI 201.1) the image is showing.
- Experimenting with more modern architectures (e.g., EfficientNet, ResNet) for faster inference or higher accuracy.
- Deploying the model as a web or mobile application for real-time crack detection.

Project Repository: https://github.com/esdeus/damage-detector-cnn.git

Technologies Used:

- Python
- TensorFlow/Keras
- VGG16
- Matplotlib
- Kaggle
- Scikit-Learn
- Numpy